Documentación del Examen

El analista de requisitos primeramente comienza analizando el proyecto integrador, leyendo atentamente los requisitos de hardware y comienza a seleccionar los componentes necesarios.

Arduino uno: este será nuestro microprocesador usando para esta práctica, dispone de varios pines E/S digitales y analógicos con varios tipos de comunicación con los que nos comunicaremos a los diferentes dispositivos

DHT22: Es un sensor digital de temperatura y humedad de bajo costo y consumo, se trata de la mejora del modelo DHT11 teniendo mayor precisión y mayor rango.

BMP280: Se trata de un sensor digital de alta precisión usando para medir presión barométrica y temperatura

LDR: Un sensor LDR (Light Depedent Resistor) es un componente electrónico cuya resistencia eléctrica varia inversamente a la intensidad de la luz(a mayor luz, menor resistencia y viceversa).

Pantalla OLED SSD1306: es un display monocromático de 128x32 pixeles, la cual usaremos para mostrar los datos recogidos de los sensores

Modulo SD: Nos permite insertar una memoria Micro SD para poder almacenar los datos de las lecturas de los sensores

Led: Un led es un componente electrónico semiconductor el cual emite una luz al paso de una corriente

Pulsador: Es un interruptor momentáneo que abre o cierra un circuito eléctrico solo mientras se mantiene presionado, regresando a su estado original al soltarlo.

Resistencias: Un resistor y resistencia es un componente electrónico pasivo diseñado para introducir una resistencia eléctrica especifica en un circuito, limitando el flujo de corriente y dividiendo voltajes

Protoboard: Es una base de plástico con orificios y laminas conductoras internas utilizado para ensamblar y probar rápidamente circuitos electrónicos

Cables Dupont: Son conductores flexibles con conectores en sus extremos, diseñados para prototipado rápido en electrónica

Buses de comunicación

Bus de datos: Transporta información real (instrucciones, datos del usuario) entre el CPU, memoria y periféricos.

Bus de dirección: Indica la posición de memoria o dispositivos con el que la CPU quiere comunicarse.

Bus de Control: Gestiona el acceso y uso de las líneas de datos y direcciones, emitiendo ordenes como lectura o escritura.

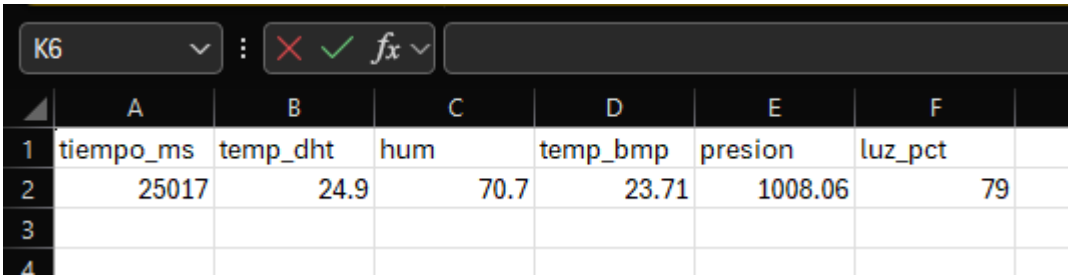
Protocolo I2C vs SPI

Tanto SPI como I2C son protocolos de comunicación serie para sistemas embebidos. Siendo SPI más rápido (10-50+MHz), es full-duplex y requiere más pines (MOSI, MISO, SCK, CS) necesitando una línea CS por cada dispositivo, pero su velocidad lo hacer adecuado para pantallas, tarjetas SD, y memorias flash. Mientras que I2C es más lento (100kHz- 3.4MHz), usa solo 2 cables (SDA/SCLI), permitiendo múltiples maestros/esclavos, haciéndolo adecuado para sensores y periféricos de baja velocidad.

A todo esto, hay que agregar que I2C verifica la integridad de los datos con un bit de acuse de recibido a diferencia de SPI no tiene un reconocimiento nativo, pero es más fácil de implementar en hardware.

Nota: el pin 13 de la placa Arduino puede ocasionar problemas a la hora de intentar usarlo debido a que esta físicamente conectado a un led integrado en la placa, esto debido a que la carga del led puede causar lecturas inestables.

Datos del archivo CSV

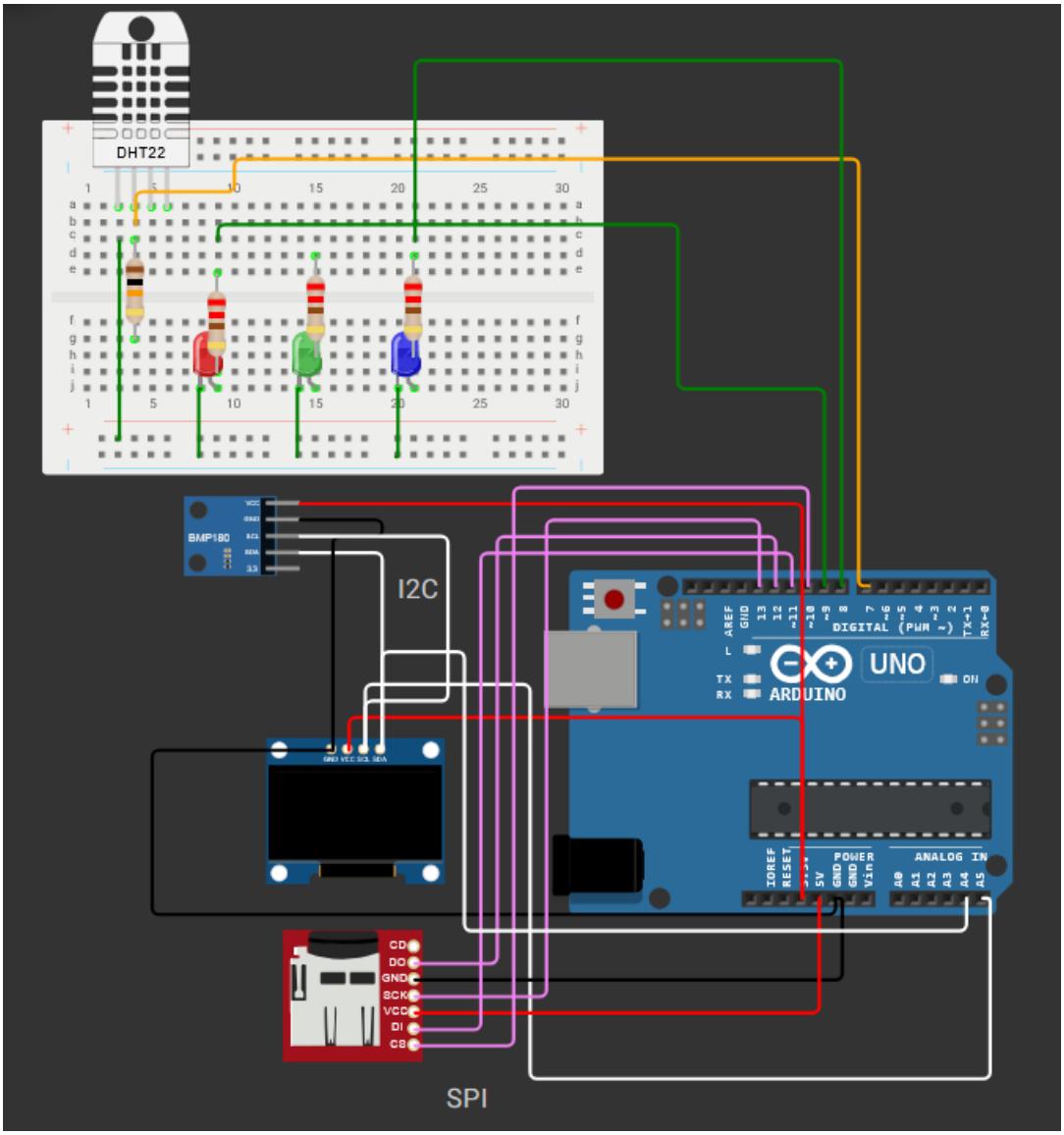


	A	B	C	D	E	F
1	tiempo_ms	temp_dht	hum	temp_bmp	presion	luz_pct
2	25017	24.9	70.7	23.71	1008.06	79
3						

Esquema

Linea I2C: Blanca

Linea SPI: Rosa



Codigo y manejo de protocolos:

En este programa se utilizan diferente protocolos de comunicación:

I2C: Pantalla OLED y Sensor BMP280

SPI: Modulo SD

UART: Para el monitor serial

Cada protocolo se maneja de forma independiente con su librería y se ejecutan en forma de secuencia, al igual se sus manejos de errores

/*

MANEJO DE ERRORES DEL SISTEMA

En este programa se utilizan variables booleanas para detectar

fallos en los diferentes componentes de la estación meteorológica.

errorDHT -> Se activa cuando el sensor DHT no responde o devuelve valores inválidos (NaN).

errorSD -> Se activa cuando hay un problema con la tarjeta SD, por ejemplo:

no se inicializa correctamente o falla la escritura del archivo.

errorBMP -> Se activa cuando el sensor BMP280 no responde o no se encuentra.

errorSensor -> Es una variable general que indica si existe algún error en el sistema.

Esta variable se construye así:

errorSensor = errorSD || errorDHT;

Si cualquiera de los sensores falla, errorSensor será TRUE, o si todos funcionan correctamente, será FALSE.

en caso de TRUE la condicional "if (errorSensor) digitalWrite(PIN_LED_ROJO, HIGH);"

indicara el fallo encendiendo el led rojo

*/

```
#include <DHT.h>
#include <Wire.h>
#include <Adafruit_BMP280.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <SD.h>
#include <SPI.h>

// ---------- Pines ESP32 ----------
#define PIN_DHT      4
#define PIN_LDR      34
#define PIN_PULSADOR  15
#define PIN_LED_VERDE 25
#define PIN_LED_AZUL  26
#define PIN_LED_ROJO  27
#define PIN_SD_CS     5

// ---------- Sensores ----------
#define DHTTYPE DHT22
DHT dht(PIN_DHT, DHTTYPE);
Adafruit_BMP280 bmp;

// ---------- OLED ----------
#define OLED_ANCHO 128
#define OLED_ALTO  64
#define OLED_RESET -1
```

```
Adafruit_SSD1306 oled(OLED_ANCHO, OLED_ALTO, &Wire, OLED_RESET);

// ---------- Tiempos ----------
unsigned long tAnteriorSensores = 0;
unsigned long tAnteriorOLED     = 0;
unsigned long tAnteriorLDR      = 0;

const long INTERVALO_SENSORES = 5000;
const long INTERVALO_LDR      = 1000;
const long INTERVALO_OLED     = 3000;

// ---------- Estado ----------
bool grabandoSD = false;
bool errorSensor = false;
byte pantallaActual = 0;
bool errorSD = false;
bool errorDHT = false;

// ---------- Datos ----------
float tempDHT = 0, humDHT = 0;
float tempBMP = 0, presionBMP = 0, altBMP = 0;
int valorLDR = 0;
float luzPct = 0;

// ---------- Botón ----------
int estadoBoton = HIGH;
```

```
int estadoBotonAnterior = HIGH;
unsigned long tRebote = 0;
const long DEBOUNCE = 50;

// ---------- Archivo ----------
const char NOMBRE_CSV[] = "/data.csv";

// ============================================================
void setup() {
  Serial.begin(115200);

  pinMode(PIN_LED_VERDE, OUTPUT);
  pinMode(PIN_LED_AZUL, OUTPUT);
  pinMode(PIN_LED_ROJO, OUTPUT);
  pinMode(PIN_PULSADOR, INPUT_PULLUP);

  // I2C (OLED + BMP)
  Wire.begin(21, 22);

  // OLED
  if (!oled.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println("Error OLED");
    errorSensor = true;
  }

  oled.clearDisplay();
```

```
oled.setTextSize(1);
oled.setTextColor(SSD1306_WHITE);

// Sensores
dht.begin();

if (!bmp.begin(0x76)) {
  Serial.println("Error BMP280");
  errorSensor = true;
}

// SD
if (!SD.begin(PIN_SD_CS)) {
  Serial.println("Error SD");
  errorSD = true;
} else {
  errorSD = false;
}

errorSensor = errorSD || errorDHT;

/*if (!SD.begin(PIN_SD_CS)) {
  Serial.println("Error SD");
  errorSensor = true;
} else {
  if (!SD.exists(NOMBRE_CSV)) {
```

```
    File archivo = SD.open(NOMBRE_CSV, FILE_WRITE);
    if (archivo) {
      archivo.println("tiempo,temp_dht,hum,temp_bmp,presion,luz");
      archivo.close();
    }
  }
}*/

  actualizarLED();
}

// ============================================================
void loop() {
  unsigned long ahora = millis();

  // BOTÓN
  int lectura = digitalRead(PIN_PULSADOR);

  if (lectura != estadoBotonAnterior) {
    tRebote = ahora;
  }

  if ((ahora - tRebote) > DEBOUNCE) {
    if (lectura != estadoBoton) {
      estadoBoton = lectura;
      if (estadoBoton == LOW) {
```

```
    grabandoSD = !grabandoSD;
    actualizarLED();
   }
  }
}

estadoBotonAnterior = lectura;

// SENSORES
if (ahora - tAnteriorSensores >= INTERVALO_SENSORES) {
  tAnteriorSensores = ahora;
  leerSensores();

  enviarSerial();

  if (grabandoSD) {
    escribirCSV();
  }
}

// LDR
if (ahora - tAnteriorLDR >= INTERVALO_LDR) {
  tAnteriorLDR = ahora;
  valorLDR = analogRead(PIN_LDR);
  luzPct = (valorLDR / 4095.0) * 100.0;
}
```

```
  // OLED
  if (ahora - tAnteriorOLED >= INTERVALO_OLED) {
    tAnteriorOLED = ahora;
    pantallaActual = (pantallaActual + 1) % 3;
    mostrarOLED();
  }
}

// ==============================================================
void leerSensores() {
  float t = dht.readTemperature();
  float h = dht.readHumidity();

  if (!isnan(t) && !isnan(h)) {
    tempDHT = t;
    humDHT = h;
  }
  if (isnan(t) || isnan(h)) {
    errorSensor = true;
  } else {
    tempDHT = t;
    humDHT = h;
    errorDHT = false;
  }
  errorSensor = errorSD || errorDHT;
```

```
  tempBMP = bmp.readTemperature();
  presionBMP = bmp.readPressure() / 100.0F;
  altBMP = bmp.readAltitude(1013.25);

  actualizarLED();
}

// ============================================================
void escribirCSV() {
  File archivo = SD.open(NOMBRE_CSV, FILE_APPEND);

  if (archivo) {
    archivo.print(millis()); archivo.print(",");
    archivo.print(tempDHT); archivo.print(",");
    archivo.print(humDHT); archivo.print(",");
    archivo.print(tempBMP); archivo.print(",");
    archivo.print(presionBMP); archivo.print(",");
    archivo.println((int)luzPct);
    archivo.close();
  }
}

// ============================================================
void mostrarOLED() {
  oled.clearDisplay();
```

```
  oled.setCursor(0, 0);

  if (pantallaActual == 0) {
    oled.println("TEMP + HUMEDAD");
    oled.print("DHT: "); oled.println(tempDHT);
    oled.print("HUM: "); oled.println(humDHT);
    oled.print("BMP: "); oled.println(tempBMP);

  } else if (pantallaActual == 1) {
    oled.println("PRESION + ALT");
    oled.print("Pres: "); oled.println(presionBMP);
    oled.print("Alt: "); oled.println(altBMP);

  } else {
    oled.println("LUZ + ESTADO");
    oled.print("Luz: "); oled.println((int)luzPct);
    oled.println(grabandoSD ? "GRABANDO" : "PAUSADO");
  }

  oled.display();
}

// ==============================================================
void actualizarLED() {
  digitalWrite(PIN_LED_VERDE, LOW);
  digitalWrite(PIN_LED_AZUL, LOW);
```

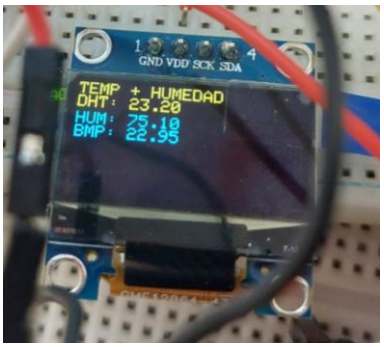
```
  digitalWrite(PIN_LED_ROJO, LOW);

  if (errorSensor) digitalWrite(PIN_LED_ROJO, HIGH);
  else if (grabandoSD) digitalWrite(PIN_LED_AZUL, HIGH);
  else digitalWrite(PIN_LED_VERDE, HIGH);
}

void enviarSerial() {
  Serial.print("Temp DHT: "); Serial.print(tempDHT);
  Serial.print(" | Hum: "); Serial.print(humDHT);
  Serial.print(" | Temp BMP: "); Serial.print(tempBMP);
  Serial.print(" | Presion: "); Serial.print(presionBMP);
  Serial.print(" | Luz: "); Serial.print((int)luzPct);
  Serial.print(" | Estado: ");
  Serial.println(grabandoSD ? "GRABANDO" : "PAUSADO");
}
```

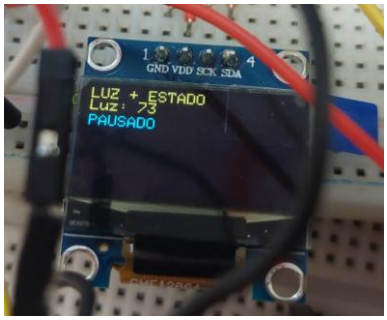
Reporte TEST / QA

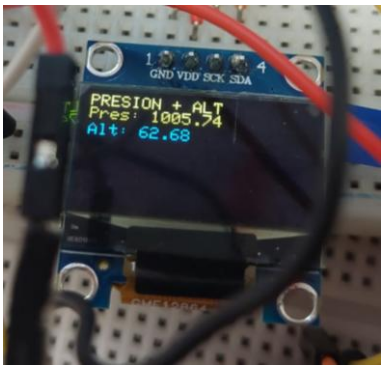
REQ-01/02: Los 3 sensores se encuentran completamente funcionales

```
Temp DHT: 22.00 | Hum: 76.80 | Temp BMP: 22.40 | Presion: 1005.73 | Luz: 78 | Estado: PAUSADO
Temp DHT: 22.10 | Hum: 77.00 | Temp BMP: 22.46 | Presion: 1005.74 | Luz: 78 | Estado: PAUSADO
Temp DHT: 22.20 | Hum: 76.90 | Temp BMP: 22.50 | Presion: 1005.69 | Luz: 78 | Estado: PAUSADO
```

REQ-03: Se verifico que la rotación de las pantallas ocurre cada 3 segundos





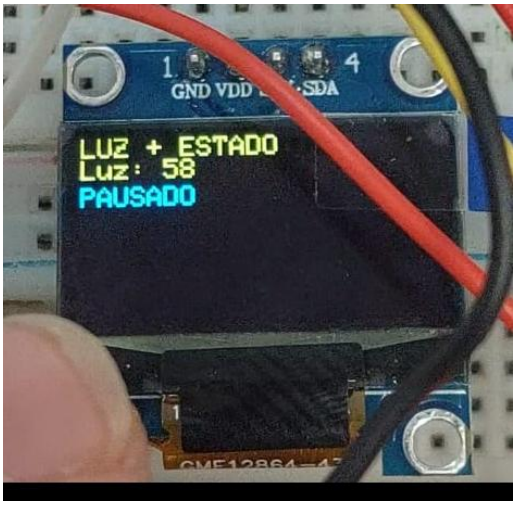


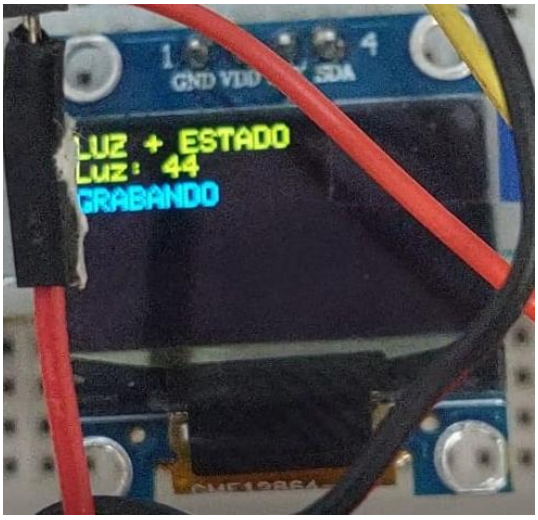
REQ-04: El programa graba correctamente los datos en un documento CSV

A1 | 15014

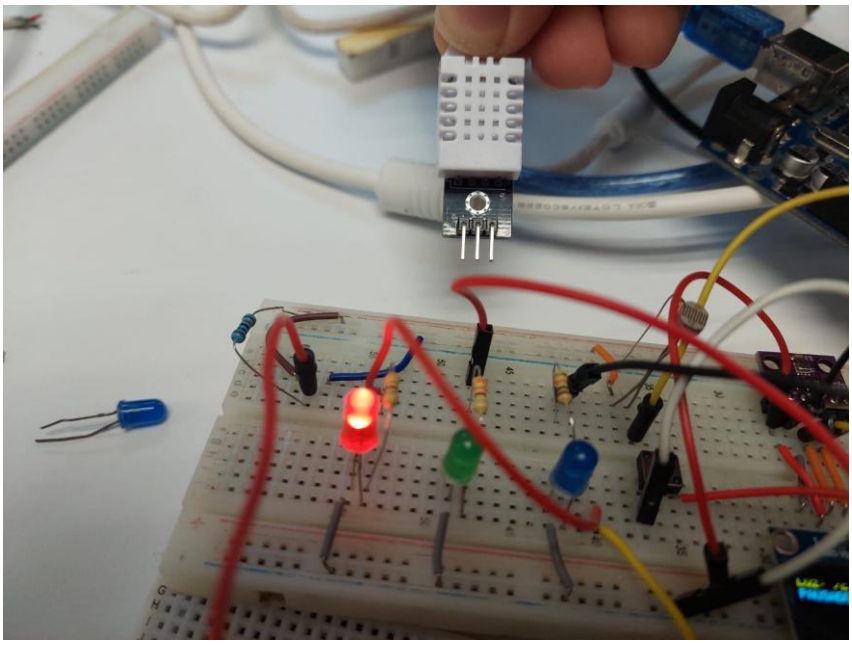
	A	B	C	D	E	F
1	15014	23.9	77.1	23.51	1005.67	78
2	20016	24	77.1	23.53	1005.72	78
3	25014	24	77.4	23.56	1005.63	78
4	30014	24.1	77.6	23.57	1005.64	78
5	35014	24.1	77.4	23.58	1005.63	78
6	40014	24.1	77.5	23.59	1005.68	78
7						

REQ-05: Se comprobó que la pantalla indica cuando se están grabando datos en la pantalla





REQ_06:Se comprobo que al desconecta algun sensor se detecta la falla de este y se enciende el led rojo



REQ-07: Se comprovo que el formato sea el correcto

```
Temp DHT: 23.80 | Hum: 77.10 | Temp BMP: 23.46 | Presion: 1005.68 | Luz: 79 | Estado: PAUSADO
Temp DHT: 23.90 | Hum: 77.10 | Temp BMP: 23.48 | Presion: 1005.69 | Luz: 79 | Estado: PAUSADO
Temp DHT: 23.90 | Hum: 77.10 | Temp BMP: 23.51 | Presion: 1005.67 | Luz: 78 | Estado: GRABANDO
```

```
void enviarSerial() {
  Serial.print("Temp DHT: "); Serial.print(tempDHT);
  Serial.print(" | Hum: "); Serial.print(humDHT);
  Serial.print(" | Temp BMP: "); Serial.print(tempBMP);
  Serial.print(" | Presion: "); Serial.print(presionBMP);
  Serial.print(" | Luz: "); Serial.print((int)luzPct);
  Serial.print(" | Estado: ");
  Serial.println(grabandoSD ? "GRABANDO" : "PAUSADO");
}
```

Caso especial: En caso de no detectarse una sd el programa arroja el mensaje “Error SD”

```
Temp DHT: 24.20 | Hum: 77.40 | Temp BMP: 23.61 | Presion: 1005.66 | Luz: 78 | Estado: PAUSADO
Temp DHT: 24.20 | Hum: 77.30 | Temp BMP: 23.61 | Presion: 1005.64 | Luz: 78 | Estado: PAUSADO
Error SD
```

Conclucion:

El desarrollo de la estación meteorológica permitió integrar múltiples sensores y módulos utilizando diferentes protocolos de comunicación como I2C, SPI y comunicación digital, logrando un sistema funcional capaz de medir, visualizar y almacenar datos ambientales. La implementación en ESP32 resolvió las limitaciones de memoria presentes en Arduino UNO, permitiendo ejecutar simultáneamente la visualización en pantalla OLED y el almacenamiento en tarjeta SD.

Además, se incorporó un sistema de control mediante botón para iniciar y detener el registro de datos, así como indicadores LED que permiten identificar el estado del sistema (funcionamiento correcto, grabación activa o presencia de errores). El manejo de errores mediante variables booleanas mejora la confiabilidad del sistema al evitar el registro de datos inválidos y facilitar la detección de fallos en sensores o en la tarjeta SD.

Como mejoras futuras, el sistema puede ampliarse significativamente para convertirlo en una solución más robusta e inteligente. Entre las principales mejoras destacan:

- **Conectividad WiFi (ESP32)**: Permitir el envío de datos a plataformas en la nube como ThingSpeak o servidores propios para monitoreo remoto en tiempo real.
- **Módulo RTC (Real Time Clock)**: Incorporar un reloj en tiempo real (como DS3231) para registrar fecha y hora exacta en los archivos CSV, incluso sin conexión a internet.
- **Almacenamiento en la nube**: Complementar o reemplazar la SD para evitar pérdida de datos y facilitar el acceso desde cualquier lugar.
- **Interfaz web o app móvil**: Visualizar gráficas y datos históricos desde un navegador o aplicación.
- **Sistema autónomo con batería/panel solar**: Hacer el sistema portátil e independiente de la energía eléctrica.

Limitaciones:
En primer lugar, la precisión de las mediciones depende directamente de la calidad y calibración de los sensores utilizados, por lo que factores como el ruido eléctrico, condiciones ambientales extremas o sensores de bajo costo pueden afectar la exactitud de los datos.

Otra limitación importante es la dependencia de la tarjeta SD para el almacenamiento local, ya que si esta falla, se desconecta o se corrompe el archivo, se pueden perder datos sin posibilidad de recuperación. Además, el sistema no cuenta con un respaldo automático ni almacenamiento alternativo.

Asimismo, el sistema carece de sincronización de tiempo real, por lo que los datos registrados no incluyen una marca de tiempo precisa, dificultando el análisis histórico detallado.